AI Masters
Алгоритмы-2024
Осень
Семестровая контрольная (midterm)
Никита Загайнов

1[3] Функции $f_i(n)$ таковы, что $\exists C > 0, N \in \mathbb{N} : \forall n > N \hookrightarrow C_1 n^3 \leq f_i(n) \leq C_2 n^5$
Найдите наилучшие оценки сверху и снизу на функцию $g(n) = \sum\limits_{i=1}^{n} f_i(n)$

Решение

$$C_1 n^3 \leq f_i(n) \leq C_2 n^5$$
$$\sum_{i=1}^{n} C_1 n^3 \leq \sum_{i=1}^{n} f_i(n) \leq \sum_{i=1}^{n} C_2 n^5$$
$$C_1 n^3 \sum_{i=1}^{n} 1 \leq g(n) \leq C_2 n^5 \sum_{i=1}^{n} 1$$
$$C_1 n^4 \leq g(n) \leq C_2 n^6$$

Таким образом, наилучшие оценки сверху и снизу для функции $g(n)$:

$$C_1 n^4 \leq g(n) \leq C_2 n^6 \Longleftrightarrow g(n) = O(n^6),\ g(n) = \Omega(n^4)$$

2[2] По некоторым современным оценкам общее число живших на Земле людей составляет около 100 миллиардов. Сколько букв должно быть в именах, чтобы никого не перепутать? Считаем, что алфавит на всех один и состоит из 34 букв.

Решение
Для существования уникального имени для каждого человека необходимо, чтобы выполнялись неравенство:

$$34^k > 100 \cdot 10^9$$

Тогда

$$k > \log_{34}(100 \cdot 10^9) \approx 7.18$$

Тогда количество букв в имени должно быть не менее

$$k \geq 8$$

3[3] На вход алгоритма поступает n-битовое число. Он печатает все нечетные числа от 1 до числа, поступившего на вход. Оцените его время работы, если печать n-битового числа занимает $O(n)$.

Решение
Рассмотрим следующий алгоритм:

```python
def print_odd_nums(n):
    cur_num = 1:
    while cur_num <= n:
        print(cur_num)
        cur_num += 2
```

Он выполняет операции сложения и вывода числа за $O(n)$, и таких операций будет $\frac{n}{2}$, таким образом, время работы алгоритма

$$O(n^2)$$

4[4] Оцените сложность возведения в n-битовую степень матрицы 2×2 с n-битовыми элементами по n-битовому модулю.

Решение

Возведение в степень матрицы 2×2 с n-битовыми элементами по n-битовому модулю эквивалентно возведению в степень двух собственных значений матрицы в её спектральном разложении:

$$A^k = Q\Lambda^k Q^{-1}$$

Так как размер матрицы фиксирован, то сложность нахождения спектрального разложения и перемножения матриц асимптотически равна самой сложной операции в этих процессах, а именно умножению чисел длины n — $O(n^2)$. Быстрое возведение применяется к константному количеству чисел, тогда сложность возведения n-битных в n-битную степень по n-битовому модулю будет асимптотически равна $O(n^3)$ Тогда общая сложность алгоритма будет равна

$$O(n^2 + n^3 + n^2) = O(n^3)$$

5[2 + 3] Для массива a существует такое i, что $a[1] < a[2] < \cdots < a[i] > a[i+1] > \cdots > a[n]$.

- Постройте алгоритм, находящий i, обоснуйте его корректность и оцените асимптотику. Найдите и обоснуйте нижнюю оценку для этой задачи.
- Постройте алгоритм, находящий j, для которого достигается максимум $|a[j] - a[j+1]|$, обоснуйте его корректность и оцените асимптотику. Найдите и обоснуйте нижнюю оценку для этой задачи.

Решение

- Эта задача может быть решена алгоритмом бинарного поиска: Построим алгоритм для нахождения i:

```python
def find_i(a):
    r = len(a) - 1
    l = 0
    while l < r:
        m = (l + r) // 2
        if a[m] < a[m + 1]:
            l = m + 1
        else:
            r = m
    return l
```

  На каждом шаге алгоритма проверяем, верно ли, что $a[mid] < a[mid+1]$, $m = \lfloor \frac{left+right}{2} \rfloor$. Если это так, то искомый элемент находится правее mid, иначе - левее. Таким образом, на каждом шаге мы снижаем диапазон поиска вдвое. Таким образом, алгоритм работает за

$$O(\log n)$$

  Докажем, что невозможно получить алгоритм, работающий быстрее, чем за $\lceil \log n \rceil$ шагов. Возьмём для примера $\lceil \log n \rceil - 1$ шагов. Тогда на каждом шаге необходимо, чтобы алгоритм отбрасывал более, чем половину элементов, но это невозможно, так как иначе есть риск отбросить меньше половины элементов (при делении не ровно пополам).

- Построим алгоритм для нахождения j:

```python
def find_j(a):
    max_diff = 0
    j = 0
    for i in range(len(a) - 1):
        if abs(a[i] - a[i + 1]) > max_diff:
            max_diff = abs(a[i] - a[i + 1])
            j = i
    return j
```

  Его асимптотика равна $O(n)$, так как он полностью проходит по массиву. Докажем, что это нижняя оценка для этой задачи. Эта задача не может быть решена быстрее, чем за $O(n)$, так как у нас нет информации о дистанции между соседними элементами, и нам необходимо рассмотреть все $n-1$ пар соседних элементов.

6[1+1+1+1+1]

- $T(n) = T(\frac{n}{2}) + T(\frac{2n}{3}) + n^2$
- $T(n) = T(n-1) + n^2$
- $T(n) = 4T(\frac{n}{16}) + \sqrt{n}$
- $T(n) = 9T(\frac{n}{3}) + n^3$
- $T(n) = 9T(\frac{n}{3}) + n$

Решение

- $T(n) = T(\frac{n}{2}) + T(\frac{2n}{3}) + n^2$

  Очевидно, что
  $$T(n) \le 2T(\frac{2n}{3}) + n^2$$
  и
  $$T(n) \ge 2T(\frac{n}{2}) + n^2$$
  Для первого соотношения по третьему случаю Мастер-теоремы получаем верхнюю оценку $\Theta(n^2)$, так как выполняется условие
  $$f(n) = \Omega(n^{\log_{1.5} 2} + \epsilon)$$
  и
  $$af(\frac{n}{b}) \le cf(n) \Longleftrightarrow 2f(\frac{n^2}{2.25}) \le \frac{8n^2}{9}$$
  Для второго случая также применим третий случай Мастер-теоремы, получаем нижнюю оценку $\Theta(n^2)$, так как выполняется условие
  $$f(n) = \Omega(n^{\log_2 2} + \epsilon)$$
  и
  $$af(\frac{n}{b}) \le cf(n) \Longleftrightarrow 2f(\frac{n^2}{4}) \le \frac{2n^2}{4}$$
  Таким образом,
  $$T(n) = \Theta(n^2)$$

- $T(n) = T(n-1) + n^2$

  $$T(n) = T(n-1) + n^2 = \sum_{i=1}^{n} i^2 \ge \frac{n}{2} \cdot \frac{n}{2}^2 = \frac{n^3}{8}\Theta(n^3)$$
  $$T(n) = \sum_{i=1}^{n} i^2 \le n \cdot n^2 = n^3 = \Theta(n^3)$$
  Таким образом, $T(n) = \Theta(n^3)$

- $T(n) = 4T(\frac{n}{16}) + \sqrt{n}$

  К задаче применим второй случай Мастер-теоремы:
  $$f(n) = \sqrt{n} = \Theta(n^{\log_b n}) = \Theta(\sqrt{n})$$
  Таким образом, $T(n) = \Theta(\sqrt{n}\log n)$

- $T(n) = 9T(\frac{n}{3}) + n^3$

  К задаче применим третий случай Мастер-теоремы:

$$f(n) = n^3 = \Omega(n^{\log_b a+\epsilon}) = \Omega(n^{2+\epsilon})$$

  и

$$af(\frac{n}{b}) \le cf(n) \Longleftrightarrow 9f(\frac{n^3}{27}) \le \frac{n^3}{3}$$

  Таким образом, $T(n) = \Theta(n^3)$

- $T(n) = 9T(\frac{n}{3}) + n$

  Применим первый случай Мастер-теоремы:

$$f(n) = n = \Theta(n^{\log_b a}) = O(n^2)$$

  Таким образом, $T(n) = \Theta(n^2)$

7[3] Оцените сложность работы алгоритма.

```
def f(n):
    for i = 0; i < n; i += 1:
        for j = 0; j < i; j += 1:
            for m = 1; m < j**2; m *= 2:
                print("hehe")

            for m = 0; m < n; m += 1:
                print("hehe")
```

Решение
Общее количество итераций равно

$$\sum_{i=0}^{n-1}\sum_{j=0}^{i-1}\left(\sum_{m=1}^{\lfloor \log j^2 \rfloor} 1 + \sum_{m=0}^{n-1} 1\right) =$$
$$\sum_{i=0}^{n-1}\sum_{j=0}^{i-1}\sum_{m=1}^{\lfloor \log (j+1)^2 \rfloor} 1 + \sum_{i=0}^{n-1}\sum_{j=0}^{i-1}\sum_{m=0}^{n-1} 1 =$$
$$\sum_{i=0}^{n-1}\sum_{j=0}^{i-1}\lfloor 2\log(j+1)\rfloor + \sum_{i=0}^{n-1}\sum_{j=0}^{i-1} n$$

Обозначим последнее выражение за $S_1(n) + S_2(n)$

$$S_1(n) \le \sum_{i=0}^{n-1}(i-1)\lfloor 2\log i\rfloor \le (n-1)(n-1)\lfloor 2\log n\rfloor = \Theta(n^2\log n)$$

$$S_1(n) \ge \sum_{i=0}^{n-1}\lfloor\frac{i-1}{2}\rfloor\lfloor 2\log\lfloor\frac{i-1}{2}\rfloor\rfloor \ge \lfloor\frac{n-1}{2}\rfloor\lfloor\frac{n-1}{2}\rfloor\lfloor 2\log\lfloor\frac{n}{2}\rfloor\rfloor = \Theta(n^2\log n)$$

и

$$S_2(n) = n\sum_{i=0}^{n-1}\sum_{j=0}^{i-1} 1 = n\sum_{i=0}^{n-1} i = n\frac{n(n-1)}{2} = \Theta(n^3)$$

Тогда

$$S_1(n) + S_2(n) = \Theta(n^2\log n) + \Theta(n^3) = \Theta(n^3)$$

8[4] Есть n монет, среди них одна фальшивая, она легче остальных, и чашечные весы. С помощью весов за один ход можно взвешивать две любые группы монет. Они могут давать три ответа: левая чашечка тяжелее, правая, равны. Постройте алгоритм, находящий фальшивую монету за асимптотически оптимальное число шагов, и докажите нижнюю оценку на эту задачу.

Решение

Рассмотрим алгоритм, который делит кучу монет на две равные части и взвесим их с помощью весов. Отбрасываем ту часть, которая тяжелее, и проводим это же действие с оставшимися монетами. Повторяем, пока не останется одна монета. Таким образом, алгоритм имеет асимптотику

$$T(n) = T(\frac{n}{2}) + O(1) = O(\log n)$$

Докажем, что это нижняя оценка для этой задачи. Каждую монету можно представить в виде битовой строки длины $\lceil \log n \rceil$, на шаге i алгоритм за константное время может может получить один бит информации, поделив оставшуюся кучу монет на две равные части. Таким образом, алгоритм должен сделать минимум $\lceil \log n \rceil$ шагов, чтобы найти фальшивую монету.

9[3] Экономный математик варит первую воду на киселе, вторую, ..., седьмую воду на киселе, ..., n-ю воду на киселе, и наливает каждую из них в банку. В каждой следующей воде на киселе концентрация киселя (действительное число) строго меньше, чем в предыдущей, и она написана на банке. Некто переставил все банки местами, и теперь они не упорядочены. Постройте алгоритм, позволяющий найти среднюю концентрацию киселя в банках с k-й воды на киселе по m-ю воду на киселе ($k < m$, нужно найти среднее по k-й, $k+1$-й, ..., m-й).

Решение

Построим следующий алгоритм, беря во внимание, что функция $median_of_medians$ за линейное время находит k-ую порядковую статистику:

```
def find_mean(k, m, a):
    kth = median_of_medians(a, k)
    mth = median_of_medians(a, m)

    running_sum = 0
    count = 0
    for elem in a:
    if elem <= kth and elem >= mth:
        running_sum += elem
        count += 1

    return running_sum / count
```

Этот алгоритм работает за $O(n)$, учитывая, что $median_of_medians$ работает за $O(n)$ и он один раз проходится по массиву.

Докажем, что асимптотика этой задачи не может быть быстрее. Примем во внимание тот факт, что нам необходимо найти среднее $m-k+1$ элементов. Это выполнимо только за $O(n)$, так как нам пройтись по всем слагаемым.